

Fase 7: Deploy Final

Verificación end-to-end + README

Tutorial paso a paso

Proyecto Sentinel AcademIA
lFunknown

20 de junio de 2026

¿Qué hicimos en este tutorial?

En la **Fase 7** (la última) hicimos el deploy final y verificamos que **todas las features funcionan end-to-end**. Este tutorial documenta:

- **Re-deploy** con todos los fixes de bugs encontrados en testing
- **Test E2E completo**: 5 endpoints probados en orden
- **Listado de todos los recursos** del stack final
- **README.md** completo para el repositorio
- **Estado final**: 50/50 tests pasando, todos los servicios activos

Hallazgos del deploy:

- **✓ 7 Lambdas** desplegadas y funcionando
- **✓ DynamoDB** con tabla multi-tenant
- **✓ SQS + DLQ** para procesamiento asíncrono
- **✓ S3** con bucket para evidencia + frontend
- **✓ API Gateway** con CORS configurado
- **✓ Frontend Vue 3** accesible públicamente
- **✓ OCI Object Storage** con knowledge base

Este es el tutorial final de la hackathon

Las 7 fases están completas. El proyecto está listo para el demo y el envío.

Índice

1. Estado del proyecto (al inicio de Fase 7)	3
1.1. Lo que teníamos	3
1.2. Lo que faltaba para “production ready”	3
2. Paso 1: Re-deploy con los fixes	3
2.1. ¿Por qué re-deploy?	3
2.2. Comando de re-deploy	3
3. Paso 2: Test E2E completo (5 pasos)	4
3.1. El test que prueba TODO	4
3.2. Output esperado (cada paso)	5

3.2.1. Paso 1: POST /api/quejas	5
3.2.2. Paso 3: PUT a S3	5
3.2.3. Paso 4: POST /api/chat	5
3.2.4. Paso 5: GET /api/quejas/id	5
3.3. ¿Qué validamos?	5
4. Paso 3: Listar todos los recursos del stack	6
4.1. Comando para listar	6
4.2. Output (resumen)	6
4.3. Outputs del stack	6
5. Paso 4: Verificar tests	7
5.1. Correr pytest	7
5.2. Output esperado	7
6. Paso 5: README.md para el repositorio	7
6.1. El README completo	7
6.2. Fragmentos clave del README	8
7. Catálogo de errores y soluciones (Fase 7)	9
8. Lo que aprendimos (resumen ejecutivo)	10
9. Siguiendo paso: tu video demo	10

1. Estado del proyecto (al inicio de Fase 7)

1.1. Lo que teníamos

Fase	Estado
0. OCI setup	[OK]
1. Backend Core	[OK]
2. LLM integration	[OK]
3. Frontend deploy	[OK]
4. File uploads	[OK]
5. RAG con OCI	[OK]
6. Testing (50/50)	[OK]
7. Deploy final	en progreso

1.2. Lo que faltaba para “production ready”

- **X**Bugs encontrados en testing (4) → aplicar fix al código deployado
- **X**Verificar E2E que todo funciona después de los fixes
- **X**README completo para el repositorio
- **X**Documentar el estado final del stack

2. Paso 1: Re-deploy con los fixes

2.1. ¿Por qué re-deploy?

En Fase 6 encontramos 4 bugs gracias a los tests:

1. `query_by_tenant` usaba `eq` en vez de `begins_with`
2. `ScanIndexForward` no soportado en `scan`
3. `SyntaxError` en `chat.py` (`\~`)
4. Imports duplicados (clases diferentes)

Aunque el bug #4 era solo de tests, los otros 3 afectaban el código de producción. Re-deployamos.

2.2. Comando de re-deploy

```

1 cd /home/lFunknown/sentinel-academia/infra
2
3 # 1. Limpiar build anterior (si hay problemas de permisos)
4 rm -rf .aws-sam-new
5
6 # 2. Build con el código actualizado
7 sam build --build-dir .aws-sam-new
8 # Output esperado:
9 #   Built Artifacts   : .aws-sam-new
10 #   Built Template    : .aws-sam-new/template.yaml
11
12 # 3. Deploy
13 sam deploy \
14   --stack-name sentinel-academia-dev \
15   --template-file .aws-sam-new/template.yaml \
16   --s3-bucket sentinel-sam-artifacts-1781951709 \

```

```

17 --capabilities CAPABILITY_IAM \
18 --no-disable-rollback \
19 --no-confirm-changeset
20 # Output esperado:
21 # Successfully created/updated stack - sentinel-academia-dev

```

Listing 1: Re-deploy con todos los fixes

3. Paso 2: Test E2E completo (5 pasos)

3.1. El test que prueba TODO

```

1 API="https://om3eiczjr9.execute-api.us-east-1.amazonaws.com/dev"
2
3 # Paso 1: Crear queja
4 echo "=== 1. POST /api/quejas ==="
5 RESP=$(curl -s -X POST "$API/api/quejas" \
6   -H "Content-Type: application/json" \
7   -H "X-Tenant-ID: demo-utpl" \
8   -H "X-Correlation-ID: final-test-001" \
9   -d '{"titulo":"Test final del proyecto",
10     "descripcion":"Esta queja prueba el flujo completo",
11     "categoriaDeclarada":"INFRAESTRUCTURA",
12     "anonima":false}')
13 echo "$RESP" | python3 -m json.tool
14 QID=$(echo "$RESP" | python3 -c "import
15     sys,json;print(json.load(sys.stdin)['quejaId'])")
16
17 # Paso 2: Pedir URL para subir archivo
18 echo "=== 2. POST /api/quejas/{id}/upload-url ==="
19 URL_RESP=$(curl -s -X POST "$API/api/quejas/$QID/upload-url" \
20   -H "Content-Type: application/json" \
21   -H "X-Tenant-ID: demo-utpl" \
22   -d '{"filename":"evidencia.pdf","contentType":"application/pdf","fileSize":2048}')
23 UPLOAD_URL=$(echo "$URL_RESP" | python3 -c "import
24     sys,json;print(json.load(sys.stdin)['uploadUrl'])")
25 echo "Upload URL: ${UPLOAD_URL:0:80}..."
26
27 # Paso 3: Subir archivo a S3
28 echo "=== 3. PUT a S3 (presigned URL) ==="
29 echo "test" > /tmp/evidencia.pdf
30 curl -s -X PUT "$UPLOAD_URL" \
31   -H "Content-Type: application/pdf" \
32   --data-binary @/tmp/evidencia.pdf \
33   -w " HTTP %{http_code}\n" -o /dev/null
34
35 # Paso 4: Hacer pregunta al chat (RAG)
36 echo "=== 4. POST /api/chat ==="
37 sleep 3
38 curl -s -X POST "$API/api/chat" \
39   -H "Content-Type: application/json" \
40   -H "X-Tenant-ID: demo-utpl" \
41   -d '{"question":"Cuales son las categorias de quejas?"}' | python3 -m
42     json.tool | head -20
43
44 # Paso 5: Verificar queja completa
45 echo "=== 5. GET /api/quejas/{id} (final) ==="

```

```
43 sleep 3
44 curl -s "$API/api/quejas/$QID" -H "X-Tenant-ID: demo-utpl" | python3 -m
    json.tool | head -20
```

Listing 2: Test E2E final completo

3.2. Output esperado (cada paso)

3.2.1. Paso 1: POST /api/quejas

```
1 {
2   "quejaId": "e096f2e6-4a73-41a6-8fca-d9c0a180589f",
3   "status": "PENDIENTE",
4   "correlationId": "final-test-001",
5   "createdAt": "2026-06-20T15:51:22.849474+00:00",
6   "estimatedAnalysisTime": 30
7 }
```

3.2.2. Paso 3: PUT a S3

```
1 HTTP 200
```

3.2.3. Paso 4: POST /api/chat

```
1 {
2   "answer": "Basado en el reglamento, las categorias son...",
3   "sources": [
4     {"id": "reglamento-cap1", "title": "Capitulo 1: ...", "score": 0.5},
5     {"id": "reglamento-cap4", "title": "Capitulo 4: ...", "score": 0.3},
6     {"id": "reglamento-faq-acoso", "title": "FAQ: ...", "score": 0.2}
7   ],
8   "modeloUsado": "cohere.command-latest",
9   "tokensUsados": 146
10 }
```

3.2.4. Paso 5: GET /api/quejas/id

```
1 {
2   "quejaId": "e096f2e6-...",
3   "status": "ANALIZADA",
4   "adjuntos": [
5     "https://sentinel-files-dev-...s3.us-east-1.amazonaws.com/..."
6   ],
7   "analysis": {
8     "categoria": "INFRAESTRUCTURA",
9     "criticidad": "MEDIA",
10    "prioridad": 5,
11    "tokensConsumidos": 25
12  }
13 }
```

3.3. ¿Qué validamos?

- ✓ Crear queja: 202 + UUID + status PENDIENTE
- ✓ Pedir presigned URL: retorna URL válida con key multi-tenant

- ✓ **Upload a S3:** 200 (sin auth, gracias a presigned URL)
- ✓ **Chat (RAG):** respuesta con sources y scores
- ✓ **Status workflow:** PENDIENTE → ANALIZADA
- ✓ **Adjuntos:** archivo subido vía S3 aparece en la queja
- ✓ **Analysis:** el mock LLM clasificó correctamente

4. Paso 3: Listar todos los recursos del stack

4.1. Comando para listar

```
1 aws cloudformation describe-stack-resources \
2   --stack-name sentinel-academia-dev \
3   --query 'StackResources[].[Type:ResourceType, Name:LogicalResourceId,
4     Id:PhysicalResourceId]' \
5   --output table
```

Listing 3: Listar recursos del stack

4.2. Output (resumen)

Tipo	Nombre	ID/ARN
AWS::Lambda::Function	HealthCheckFunction	sentinel-health-dev
AWS::Lambda::Function	CreateQuejaFunction	sentinel-create-queja-dev
AWS::Lambda::Function	GetQuejaFunction	sentinel-get-queja-dev
AWS::Lambda::Function	ListQuejasFunction	sentinel-list-quejas-dev
AWS::Lambda::Function	AnalyzeQuejaFunction	sentinel-analyze-queja-dev
AWS::Lambda::Function	UploadURLFunction	sentinel-upload-url-dev
AWS::Lambda::Function	FileProcessorFunction	sentinel-file-processor-dev
AWS::Lambda::Function	ChatFunction	sentinel-chat-dev
AWS::DynamoDB::Table	QuejasTable	sentinel-quejas-dev
AWS::SQS::Queue	AnalysisQueue	sentinel-analysis-dev
AWS::SQS::Queue	AnalysisDeadLetterQueue	sentinel-analysis-dlq-dev
AWS::S3::Bucket	FilesBucket	sentinel-files-dev-227165337884
AWS::ApiGateway::RestApi	SentinelApi	om3eiczjr9

4.3. Outputs del stack

```
1 aws cloudformation describe-stacks \
2   --stack-name sentinel-academia-dev \
3   --query 'Stacks[0].Outputs' \
4   --output table
```

Listing 4: Obtener outputs

```
1 -----
2 |  OutputKey          |  OutputValue
3 |-----|-----
4 |  ApiUrl              |
5 |  https://om3eiczjr9.execute-api.us-east-1.amazonaws.com/dev
6 |  QuejasTableName     |  sentinel-quejas-dev
7 |  AnalysisQueueUrl    |
8 |  https://sqs.us-east-1.amazonaws.com/.../sentinel-analysis-dev
```

```

7 | FilesBucketName | sentinel-files-dev-227165337884
8 | FrontendUrl |
  http://sentinel-frontend-dev-...s3-website-us-east-1.amazonaws.com
9 -----

```

Listing 5: Output esperado

5. Paso 4: Verificar tests

5.1. Correr pytest

```

1 cd /home/lfunkown/sentinel-academia
2 export PYTHONPATH=src:$PYTHONPATH
3 python3 -m pytest src/tests/ -v

```

Listing 6: Verificar 50/50 tests pasando

5.2. Output esperado

```

1 src/tests/unit/test_shared.py::TestConfig::test_default_values PASSED
2 src/tests/unit/test_shared.py::TestConfig::test_dynamodb_table_resolved
  PASSED
3 src/tests/unit/test_shared.py::TestCorrelationId::test_with_correlation_id
  PASSED
4 ... (47 more)
5 src/tests/integration/test_handlers.py::TestChat::test_chat_requires_tenant
  PASSED
6
7 ===== 50 passed in 2.09s
  =====

```

6. Paso 5: README.md para el repositorio

6.1. El README completo

El README.md en la raíz del proyecto incluye:

- **Arquitectura** con diagrama ASCII
- **Stack tecnológico** completo (Python, AWS, OCI, Vue)
- **Estructura del proyecto** (todos los archivos)
- **Setup local** paso a paso
- **Deploy en AWS** con comandos
- **Configuración de OCI** con Python SDK
- **Endpoints de la API** con ejemplos curl
- **Tests** cómo correrlos
- **Troubleshooting** de errores comunes
- **Stack final** desplegado (todos los recursos)

6.2. Fragmentos clave del README

```
1 ## Setup local
2
3 ### Requisitos
4 - Python 3.12+
5 - Node.js 18+
6 - AWS CLI 2.x
7 - SAM CLI 1.162+
8 - mise 2026+
9 - Tectonic 0.16+ (para compilar PDFs)
10
11 ### 1. Clonar
12 git clone https://github.com/your-org/sentinel-academia.git
13 cd sentinel-academia
14
15 ### 2. Python 3.12
16 mise use --global python@3.12
17
18 ### 3. Deps Python
19 pip install pydantic pydantic-settings email-validator boto3 \
20     aws-lambda-powertools[tracer,logger,metrics] orjson \
21     pytest pytest-cov pytest-mock 'moto[dynamodb,s3,sqs]' \
22     boto3-stubs[dynamodb,s3,sqs]
23
24 ### 4. Deps Node
25 cd web && npm install && cd ..
26
27 ### 5. Credenciales AWS Academy
28 export AWS_ACCESS_KEY_ID="ASIAT..."
29 export AWS_SECRET_ACCESS_KEY="..."
30 export AWS_SESSION_TOKEN="IQoJb3JpZ2luX2Vj..."
31 export AWS_DEFAULT_REGION="us-east-1"
32 export AWS_REGION="us-east-1"
33 aws sts get-caller-identity
```

Listing 7: README.md Instalacion

```
1 ## Deploy en AWS
2
3 ### Build
4 cd infra
5 sam build --build-dir .aws-sam-new
6
7 ### Deploy
8 sam deploy \
9     --stack-name sentinel-academia-dev \
10    --template-file .aws-sam-new/template.yaml \
11    --s3-bucket YOUR-SAM-ARTIFACTS-BUCKET \
12    --capabilities CAPABILITY_IAM \
13    --no-disable-rollback \
14    --no-confirm-changeset
15
16 ### S3 notification (post-deploy)
17 BUCKET=$(aws cloudformation describe-stacks \
18     --stack-name sentinel-academia-dev \
19     --query
20     'Stacks[0].Outputs[?OutputKey=='FilesBucketName'].OutputValue' \
21     --output text)
```



```
21
22 LAMBDA_ARN=$(aws lambda get-function \
23   --function-name sentinel-file-processor-dev \
24   --query 'Configuration.FunctionArn' --output text)
25
26 aws s3api put-bucket-notification-configuration \
27   --bucket "$BUCKET" \
28   --notification-configuration "{...}"
```

Listing 8: README.md Deploy

7. Catálogo de errores y soluciones (Fase 7)

Error 1: Tests pasan pero deploy falla

Cuando: pytest OK pero el sistema en prod no.

Causa: tests usan moto (mock), prod usa AWS real.

Solución: hacer test E2E con curl al API real.

Error 2: CORS preflight fails

Cuando: el frontend no puede llamar al API.

Causa: API Gateway no tiene headers CORS.

Solución: verificar `Globals.Api.Cors` en el template.

Error 3: S3 notification no dispara

Cuando: subes archivo pero la queja no se actualiza.

Causa: notification config no aplicada.

Solución: `aws s3api get-bucket-notification-configuration`.

Error 4: Stack en ROLLBACK_COMPLETE

Cuando: deploy falla y no se puede re-deployar.

Solución:

```
1 aws cloudformation delete-stack --stack-name sentinel-academia-dev
2 aws cloudformation wait stack-delete-complete --stack-name
   sentinel-academia-dev
3 sam deploy ...
```

8. Lo que aprendimos (resumen ejecutivo)

Checklist Fase 7

1. **Re-deploy** después de tests para aplicar fixes
2. **Test E2E** con curl (5 pasos secuenciales)
3. **Verificar** todos los outputs del stack
4. **pytest** sigue verde después del re-deploy
5. **README.md** completo para el repositorio
6. **Documentar** el estado final (todos los recursos)
7. **Tests pasan** (50/50)
8. **7 fases** completas
9. **Hackathon** lista para el demo
10. **Siguiente:** el usuario graba el video demo

9. Siguiente paso: tu video demo

El usuario grabará el video demo. Aquí hay un guión sugerido:

1. **Intro (15s):** “Soy [nombre], esto es Sentinel AcademIA, una plataforma de quejas universitarias con IA. Hackathon 2026.”
2. **Arquitectura (30s):** mostrar el diagrama en el README. Explicar AWS + OCI.
3. **Demo en vivo (90s):**
 - Abrir el frontend (<http://sentinel-frontend-dev-...s3-website...>)
 - Crear una queja: “Problema con el aula 205”
 - Esperar 5 segundos
 - Mostrar el análisis: categoría INFRAESTRUCTURA, criticidad MEDIA
 - Hacer una pregunta al chat: “¿Qué hago si sufro acoso?”
 - Mostrar la respuesta con sources del reglamento
4. **Backend (30s):** mostrar la consola de AWS:
 - CloudWatch logs de la Lambda
 - DynamoDB con la queja creada
 - S3 con el archivo de evidencia
5. **Tests (15s):** mostrar **pytest** corriendo 50 tests en 2 segundos.
6. **Cierre (15s):** “50 tests, 7 Lambdas, multi-tenant, RAG, y todo deployado en AWS Academy. Gracias.”

Fin del tutorial. Éxito en tu demo!